

Disciplina: Projeto e Análise de Algoritmos

Aluno: Eduardo Lopes Jonker

Linguagem: Python

Explicação Lógica do Algoritmo

O problema de percorrer uma árvore binária é resolvido através de **Busca em Profundidade (DFS)**. A recursão é a ferramenta ideal aqui porque permite que o programa "empilhe" as chamadas de cada nó e retorne exatamente para onde parou após explorar um caminho até o fim.

1. A Estrutura Base (Class No)

O código define um objeto que armazena três informações essenciais: o **valor** do dado, uma referência para o filho da **esquerda** e outra para o da **direita**. Se um filho for None, significa que chegamos a uma folha (o fim daquele ramo).

2. Mecanismo de Recursão

As três funções (pre_ordem, em_ordem, pos_ordem) funcionam de forma quase idêntica, alterando apenas a **posição da visita** (o momento em que o valor do nó é impresso).

- **Pré-ordem (V-E-D):** O algoritmo visita a raiz **antes** de descer para os filhos. É como se ele marcasse o território assim que chega no nó.
 - *Resultado no exemplo:* Começa no 4, vai para o 2, depois 1...
- **Em-ordem (E-V-D):** O algoritmo desce o máximo possível para a esquerda. Ele só "visita" o nó quando não há mais nada para a esquerda ou quando volta de lá. Em Árvores Binárias de Busca (BST), isso resulta nos números em ordem crescente.
 - *Resultado no exemplo:* O 1 é o primeiro a ser impresso, pois é o mais à esquerda.
- **Pós-ordem (E-D-V):** O algoritmo explora toda a linhagem de filhos (esquerda e direita) e só visita a raiz por último. É o método usado, por exemplo, para deletar uma árvore da memória com segurança (você apaga os filhos antes do pai).

3. Complexidade de Algoritmo

Do ponto de vista de análise:

- **Complexidade de Tempo:** $O(n)$, onde n é o número de nós, pois cada nó é visitado exatamente uma vez.
- **Complexidade de Espaço:** $O(h)$, onde h é a altura da árvore, representando a quantidade máxima de chamadas na pilha de execução (stack).

Demonstração do Fluxo

No seu exemplo, quando chamamos `em_ordem(raiz)`:

1. O código recebe o **4**.
2. Ele não imprime o 4; ele chama a função para o filho **2**.
3. No **2**, ele chama a função para o filho **1**.
4. No **1**, ele tenta ir para a esquerda (vazio), volta, **imprime o 1**, tenta ir para a direita (vazio) e encerra.
5. A execução volta para o nó **2**, que agora é impresso, e segue para o **3**.

Esse efeito dominó garante que todos os nós sejam processados na ordem exata solicitada.

```
class No:
def __init__(self, valor):
self.valor = valor
self.esquerda = None
self.direita = None

def pre_ordem(raiz):
# Raiz -> Esquerda -> Direita
if raiz:
print(raiz.valor, end=", " if raiz.valor != 9 else "")
pre_ordem(raiz.esquerda)
pre_ordem(raiz.direita)

def em_ordem(raiz):
# Esquerda -> Raiz -> Direita
if raiz:
em_ordem(raiz.esquerda)
print(raiz.valor, end=", " if raiz.valor != 9 else "")
em_ordem(raiz.direita)

def pos_ordem(raiz):
# Esquerda -> Direita -> Raiz
if raiz:
pos_ordem(raiz.esquerda)
pos_ordem(raiz.direita)
```

```
print(raiz.valor, end=" ", " if raiz.valor != 4 else ""))
```

```
# Montando a árvore do seu exemplo
```

```
raiz = No(4)
```

```
raiz.esquerda = No(2)
```

```
raiz.direita = No(8)
```

```
raiz.esquerda.esquerda = No(1)
```

```
raiz.esquerda.direita = No(3)
```

```
raiz.direita.esquerda = No(6)
```

```
raiz.direita.direita = No(9)
```

```
raiz.direita.esquerda.esquerda = No(5)
```

```
raiz.direita.esquerda.direita = No(7)
```

```
# Execução
```

```
print("Pré-ordem:", end=" ")
```

```
pre_ordem(raiz)
```

```
print("\nEm-ordem: ", end=" ")
```

```
em_ordem(raiz)
```

```
print("\nPós-ordem:", end=" ")
```

```
pos_ordem(raiz)
```

Pré-ordem: 4, 2, 1, 3, 8, 6, 5, 7, 9

Em-ordem: 1, 2, 3, 4, 5, 6, 7, 8, 9

Pós-ordem: 1, 3, 2, 5, 7, 6, 9, 8, 4(base) eduardo-note@eduardonote-Insp

iron-15-3530:~/Documentos/Algoritmo-05-05\$